If you had to do one today, which one?

clicker.cs.illinois.edu

Q1

~Code~
340

CS 340

C without the ++ (0b10)

Updates

1. MP 1.1, MP 1.2
2. Environment check-off
3. HW 2 is out
4. Collaboration time/office hours today

C without the ☝++

LGs:

- Be able to read and understand C code
- Be able to write C code from scratch

1. Review ←
2. Dynamic memory
3. Using arrays
4. C-strings
5. Structs
6. BST example

What prints on line 6?

clicker.cs.illinois.edu

Q2

~Code~
340

```
3    int main(){
4        char arr[3];
5        printf("%x\n", arr);
6        printf("%x\n", arr+2);
7    }
```

1 byte

0x1306fab980

arr + (1)(2)

↑

size of a char

0x1306fab982

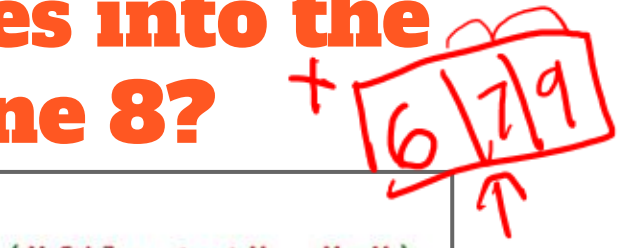
How many bytes into the file is fl after line 8?

clicker.cs.illinois.edu

Q3

~Code~ 340

```
3   int main(){
4       FILE *fl = fopen("file.txt", "r");
5       int x[3];
6       int read = fread(x, 4, 1, fl);
7       fseek(fl, 8, SEEK_SET);
8       int read = fread(x+1, 4, 2, fl);
9       fclose(fl);
10  }
```



SEEK_SET → start

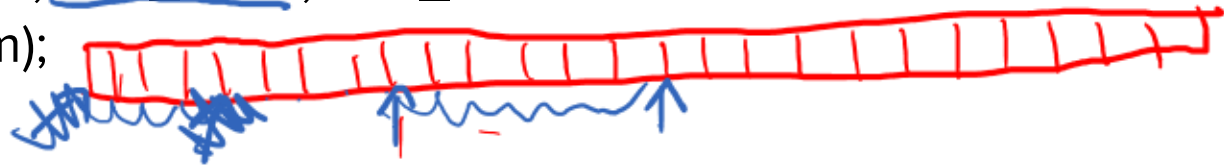
SEEK_CUR → current

SEEK_End → end

16

int fseek(FILE *stream, long int offset, int whence);

size_t fread(void *restrict ptr, size_t size, size_t nitems, FILE *restrict stream);



no new, delete

Dynamic Memory

malloc free

[Review] What is Dynamic Memory

Memory the program requests at ..... run time

- Allows – flexibility for memory
- Examples
  - grow a data structure
    - array
    - linked-list
    - tree

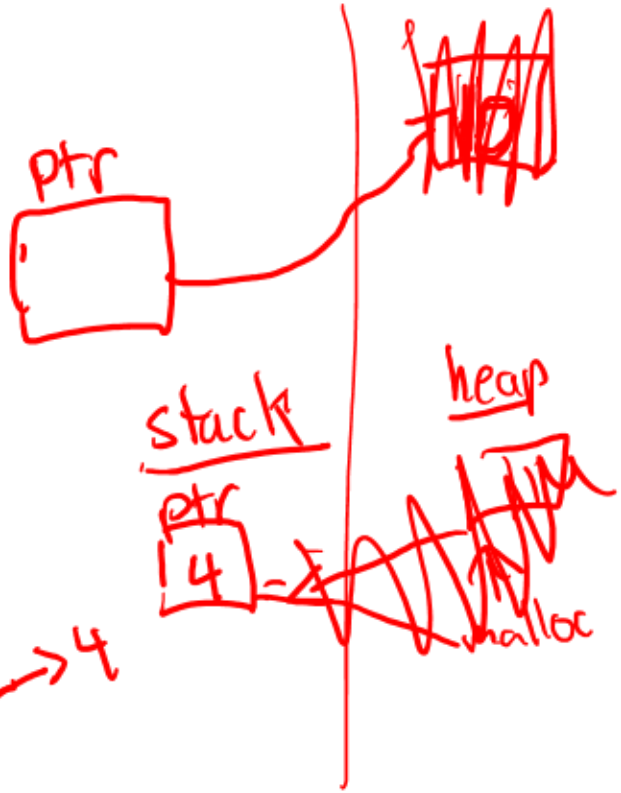
We control the lifetime of the requested memory ← not scope

//allocates size bytes on the heap and returns a
//pointer to that memory location on the heap.
void *malloc(size_t size);

//frees the memory at ptr from the heap
void free(void *ptr);

```c
#include <stdlib.h>

int main(){
    int *ptr = malloc(sizeof(int));
    *ptr = 10;
    free(ptr);
}
```



//resizes a previously allocated dynamic memory block

void *realloc(void *ptr, size_t new_size);

//creates num*size memory initialized to 0

void* calloc(size_t num, size_t size);

```
1   #include <stdlib.h>
2
3   int main(){
4       int *ptr = calloc(3, sizeof(int));
5       ptr[2] =ptr[2] + 6;
6       ptr = realloc(ptr, sizeof(int)*4);
7       ptr[3] = 100;
8       free(ptr);
9   }
```

What could the following print?

2 2 0x4ab5

clicker.cs.illinois.edu

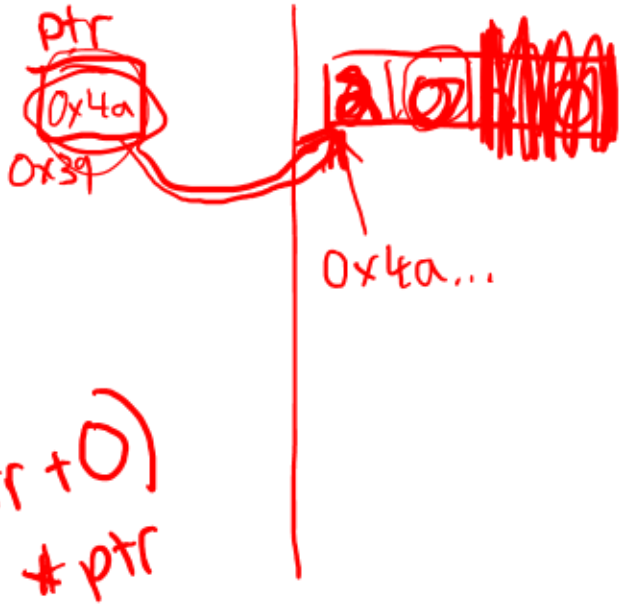
Q4

~Code~
340

```
4   int main(){
5       int *ptr = calloc(2, 4);
6       *ptr = ptr[1] + 2;
7       printf("%i\n", *ptr);
8       printf("%i\n", ptr[0]);
9       printf("%x\n", ptr);
10      free(ptr);
11  }
```

same

*(ptr + 0)
*ptr



What is wrong?

clicker.cs.illinois.edu

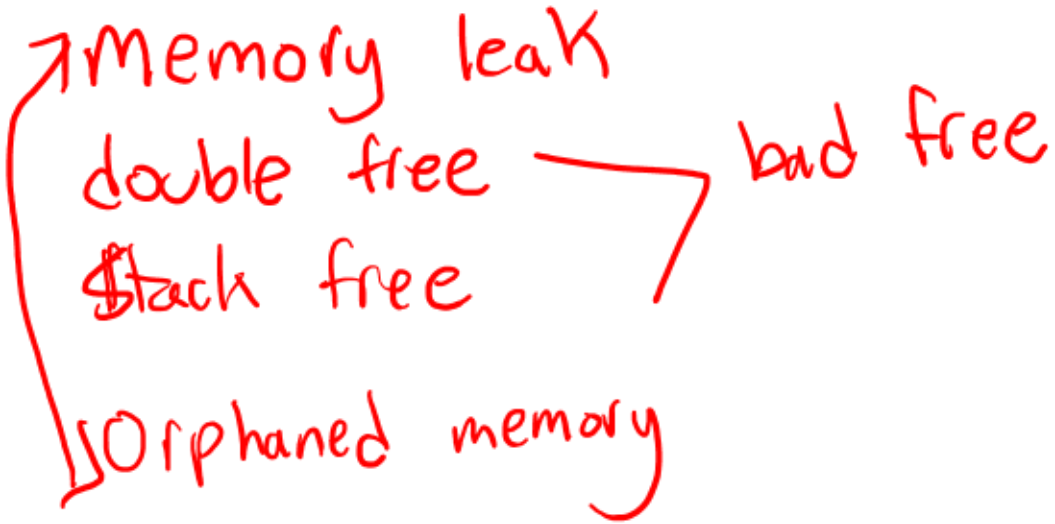
Q5

~Code~
340

```
1    #include <stdlib.h>
2
3    int main(){
4        int x = 5;
5        int *ptr = malloc(sizeof(int));
6        ptr = &x;
7        free(ptr);
8    }
```

memory leak + bad free

[Review] What are some common memory management errors?



Finding Memory Errors

MP 1.2

valgrind --leak-check=full --show-leak-kinds=all ./exec

```
==10389== HEAP SUMMARY:
==10389==     in use at exit: 24 bytes in 1 blocks
==10389==   total heap usage: 1 allocs, 0 frees, 24 bytes allocated
==10389==
==10389== 24 bytes in 1 blocks are definitely lost in loss record 1 of 1
==10389==    at 0x488545C: malloc (vg_replace_malloc.c:446)
==10389==    by 0x4006C3: add_node_helper (bst.c:21)
==10389==    by 0x40076B: add_node (bst.c:37)
==10389==    by 0x40080B: main (bst.c:58)
==10389==
==10389== LEAK SUMMARY:
==10389==    definitely lost: 24 bytes in 1 blocks
==10389==    indirectly lost: 0 bytes in 0 blocks
==10389==      possibly lost: 0 bytes in 0 blocks
==10389==    still reachable: 0 bytes in 0 blocks
==10389==         suppressed: 0 bytes in 0 blocks
==10389==
==10389== For lists of detected and suppressed errors, rerun with: -s
==10389== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

How would I read 2 ints into buffer from file.txt?

clicker.cs.illinois.edu

Q6

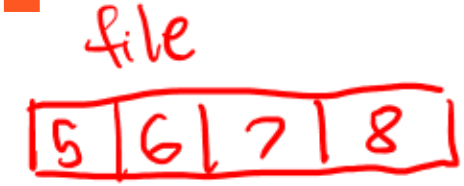
~Code~
340

```
3   int main(){
4       FILE* fl = fopen("file.txt", "r");
5       int* buffer = malloc(400);
6       fread(                          );
7       //do some other logic
8       free(buffer);
9   }
```

(buffer, 2, 4, fl)

(void * ptr, num items, size, FILE* p)

How do I read in 1 int into buffer at byte 40?

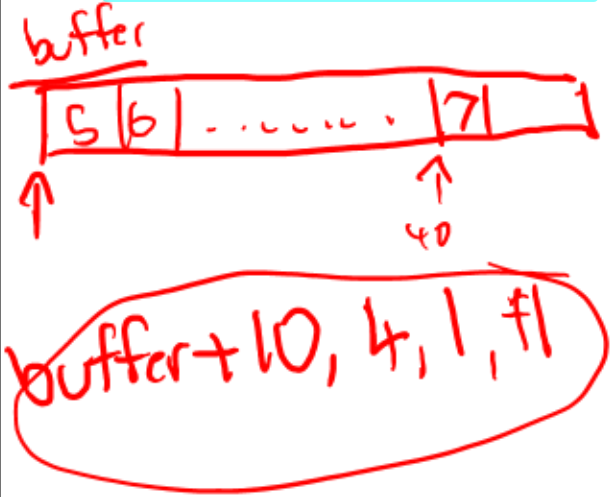


clicker.cs.illinois.edu

Q7

~Code~
340

```
3   int main(){
4       FILE* fl = fopen("file.txt", "r");
5       int* buffer = malloc(400);
6       fread(buffer, 4, 2, fl);
7       //do some logic
8       fread(                              );
9       //do some other logic
10      free(buffer);
11  }
```



Using Arrays - Grow - Functions

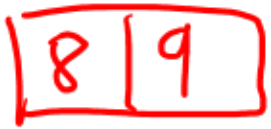
Arrays are limited

```
10    int main() {
11        int arr[2];
22    }
```

don't know size
can't grow

Arrays are limited

```
10  int main() {
11      int arr[2];
12      int size = 0;
13
14      arr[size] = 8;
15      size++;
16
17      arr[size] = 9;
18      size++;
19
20      arr[size] = 10;
21      size++;
22  }
```



Growing an Array

```
10  int main() {
11      int cap = 2;
12      int* arr = malloc(cap*sizeof(int));
13      int size = 0;
14
15      arr[0] = 1;
16      arr[1] = 2;
17      size = 2;
18
19      if(size == cap){
20          arr = realloc(arr, cap*2);
21          cap = cap*2;
22      }
23      arr[size] = 3;
24  }
```

What goes in the ??

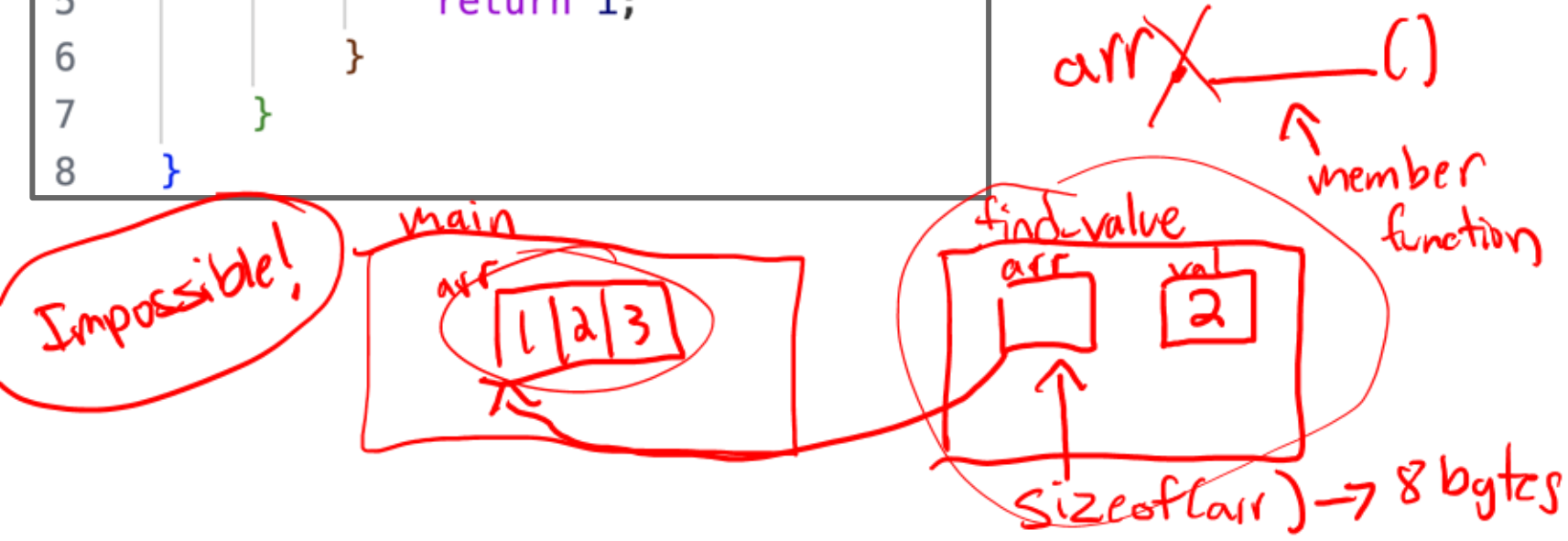
```
2    int find_value(int arr[], int val){
3        for(int i = 0; i < ??; i++){
4            if(arr[i] == val){
5                return i;
6            }
7        }
8    }
```

int * arr

clicker.cs.illinois.edu

Q8

~Code~ 340



Arrays with functions

```
2   int find_value(int arr[], int val){
3       for(int i = 0; i < ??; i++){
4           if(arr[i] == val){
5               return i;
6           }
7       }
8   }
```

```
2   int find_value(int arr[], int size, int val){
3       for(int i = 0; i < size; i++){
4           if(arr[i] == val){
5               return i;
6           }
7       }
8   }
```

C-Strings

char

1 byte == 8 bits = decimal values 0-255

ASCII - mapping from decimal to character

'a' → 97

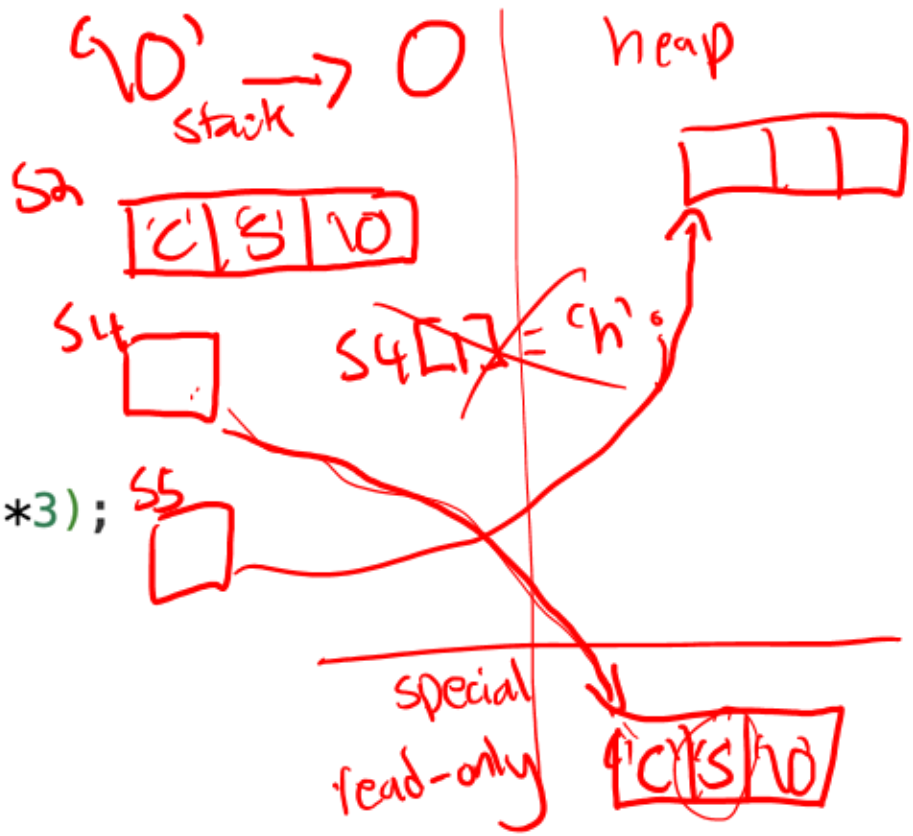
* char

C-Strings/char* ← array of chars

```
char s3[3] = {'C','S','\0'};
char s2[3] = "CS";
char* s4 = "CS";
char* s5 = malloc(sizeof(char)*3);
```

← copy

3 bytes



What prints?

clicker.cs.illinois.edu

Q9

~Code~ 340

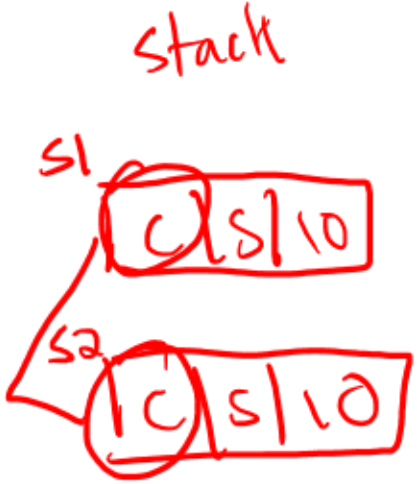
```
15  int main() {
16      char s1[3] = {'C', 'S', '\0'};
17      char s2[3] = {'C', 'S', '\0'};
18      if (s1 == s2){
19          printf("yay");
20      }
21  }
```

nothing

What prints?

```
15    int main() {
16        char s1[3] = "CS";
17        char s2[3] = "CS";
18        if (s1 == s2){
19            printf("yay");
20        }
21    }
```

copy CS\0 into s1



nothing

clicker.cs.illinois.edu

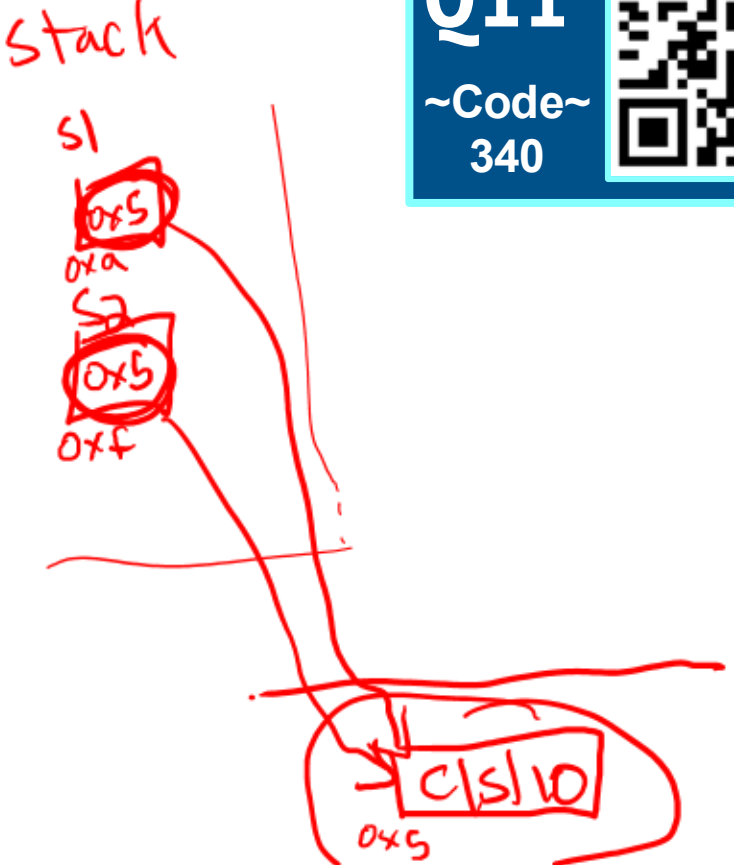
Q10

~Code~
340

What prints?

```
15  int main() {
16      char *s1 = "CS";
17      char *s2 = "CS";
18      if (s1 == s2){
19          printf("yay");
20      }
21  }
```

yay



clicker.cs.illinois.edu

Q11

~Code~
340

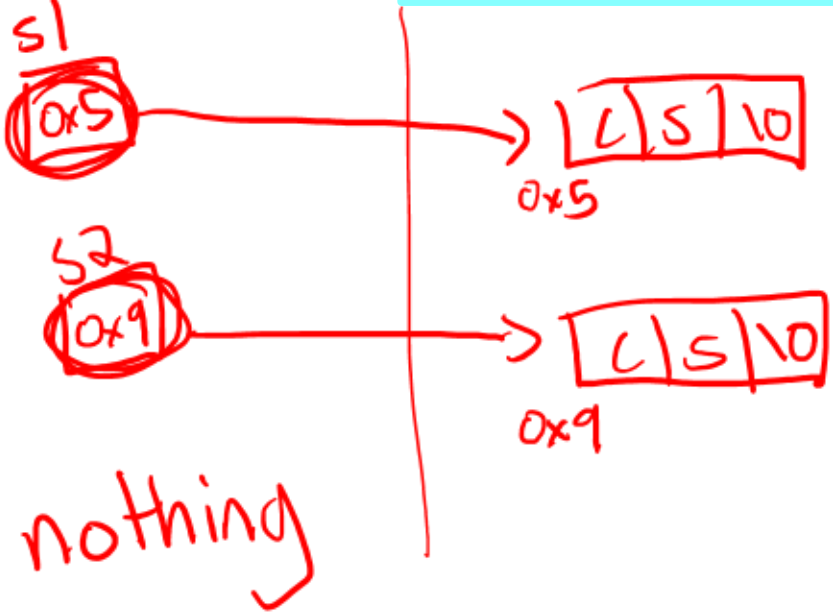
What prints?

```
3   int main(){
4       char *s1 = malloc(3);
5       s1[0] = 'C';
6       s1[1] = 'S';
7       s1[2] = 0;
8       char *s2 = malloc(3);
9       s2[0] = 'C';
10      s2[1] = 'S';
11      s2[2] = 0;
12      if (s1 == s2){
13          printf("yay");
14      }
15  }
```

clicker.cs.illinois.edu

Q12

~Code~
340



#include <string.h>

size_t strlen(char *str)

char *strcpy(char *dest, const char *src)

int strcmp(const char *str1, const char *str2)

0 if equal

What is the issue with the top code snippet?

```
15    int main() {
16        char *s1 = malloc(sizeof(char)*3);
17        s1 = "CS";
18        free(s1);
19    }
```

Solution!

```
15  #include <string.h>
16
17  int main() {
18      char *s1 = malloc(sizeof(char)*3);
19      strcpy(s1, "CS");
20      free(s1);
21  }
```

no classes
no member functions

Structs

w/ member variables

public

Is this valid C code?

```
4    struct food {
5        int amount;
6        int age;
7
8        int can_eat(){
9            if(amount > 0 && age < 10){
10               return 1;
11           }
12           return 0;
13       }
14   };
```

clicker.cs.illinois.edu

Q13

~Code~
340

Working with Structs in C

Instead of making a class with member functions...

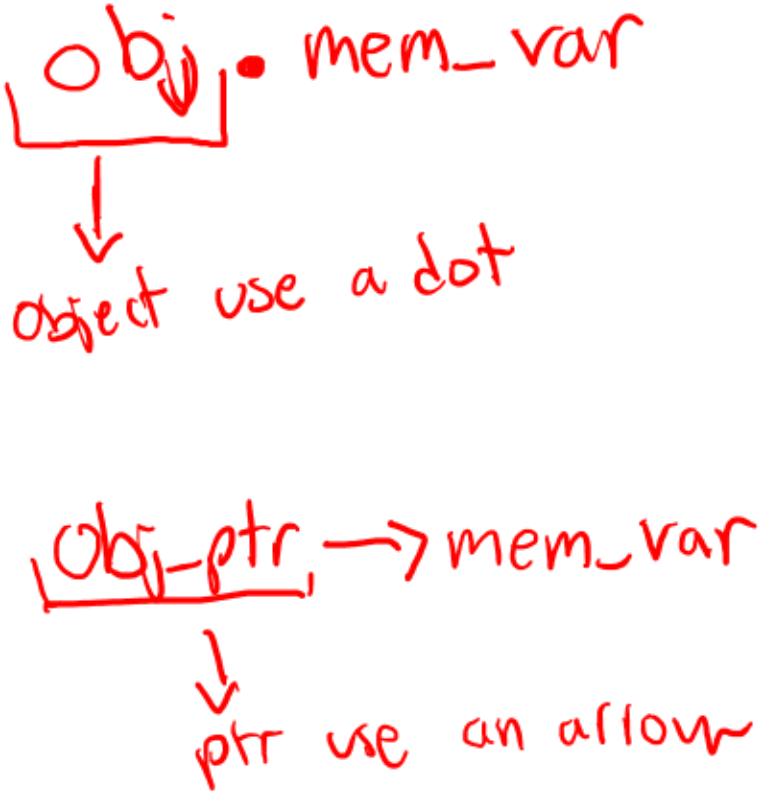
Working with Structs in C Example

```
2   typedef struct food {
3       int amount;
4       int age;
5   } food;
6
7   int can_eat(food *self){
8       if(self->amount > 0
9           && self->age < 10){
10          return 1;
11      }
12      return 0;
13  }
```

pointer to a food object

```
14  int main(){
15      food test;
16      test.amount = 1;
17      test.age = 13;
18      int val = can_eat(&test);
19      printf("%i\n", val);
20  }
```

[Review] -> versus .



What would go in each box?

```
 4    typedef struct food {
 5        int amount;
 6        int *age;
 7    } food;
 8
 9    int main(){
10        int x = 5;
11        food fd;
12        fd[ ]amount = 2;
13        fd[ ]age = &x;
14        food *fd_p = &fd;
15        *(fd_p[ ]age) = 7;
16    }
```

clicker.cs.illinois.edu

Q14

~Code~
340

```
4    typedef struct food {
5        int amount;
6        int age;
7    } food;
8
9    void food_init(food *self, int am){
10       self->amount = am;
11       self->age = 0;
12   }
```

How to it create and initialize a food object?

clicker.cs.illinois.edu

Q15

~Code~ 340

```
int main(){
    food fd = food_init(&fd, 2);
}
```

```
int main(){
    food fd;
    food_init(&fd, 2);
}
```

```
int main(){
    food fd(2);
}
```

BST Example

What goes in the box?

clicker.cs.illinois.edu

Q16

~Code~
340

```
5   typedef struct node {
6     struct node *left;
7     struct node *right;
8     int datum;
9   } node;
10
11  typedef struct bst {
12    struct node *root;
13  } bst;
14
15  void init_bst(bst *self) {
16    [redacted]
17  }
```

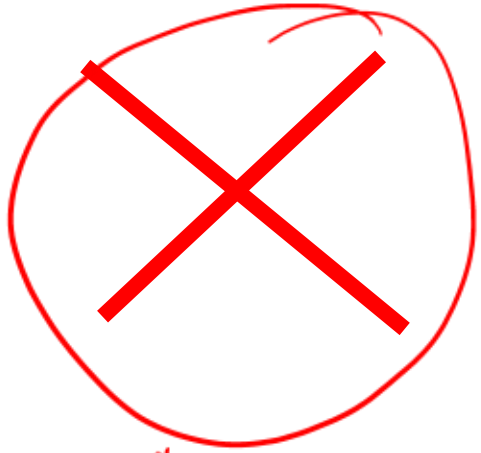
self->root = NULL

```c
typedef struct node {
  struct node *left;
  struct node *right;
  int datum;
} node;

typedef struct bst {
  struct node *root;
} bst;

void init_bst(bst self){
    self.root = NULL;
}
```

```c
int main() {
  bst b;
  init_bst(b);
  return 0;
}
```

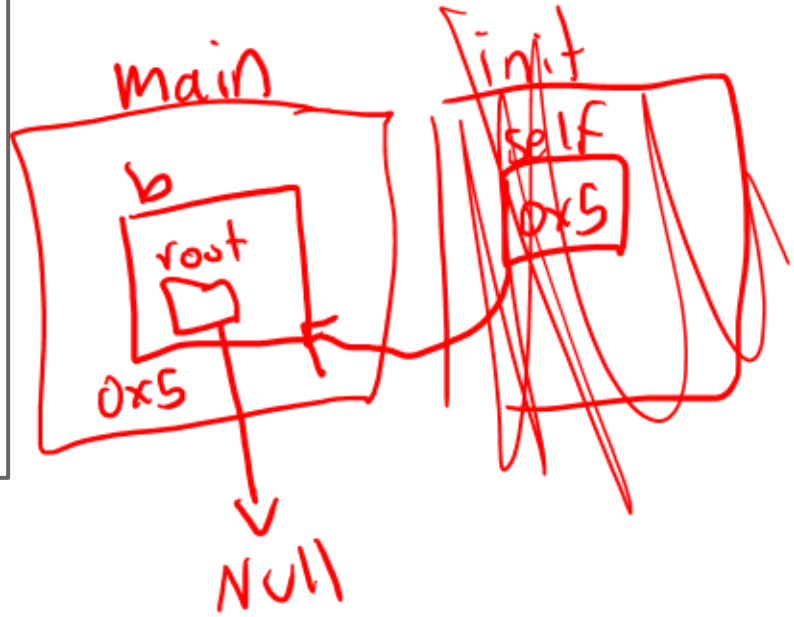


```c
typedef struct node {
  struct node *left;
  struct node *right;
  int datum;
} node;

typedef struct bst {
  struct node *root;
} bst;

void init_bst(bst *self){
  self->root = NULL;
}
```

```c
int main() {
    bst b;
    init_bst(&b);
    return 0;
}
```



What goes in the box?

clicker.cs.illinois.edu

Q17

~Code~
340

```
void add_node(bst *self, int data){
    self->root = add_node_helper(self->root, data);
}
```

```
19  node *add_node_helper(node *node_ptr, int data){
20      if(node_ptr == NULL){
21          node *tmp = malloc(          );
22          tmp->left = NULL;
23          tmp->right = NULL;
24          tmp->datum = data;
25          return tmp;
26      }
```

sizeof(node)

more

Given the code so far, does bst have any member functions?

clicker.cs.illinois.edu

Q18

~Code~
340

Is the default constructor called?

clicker.cs.illinois.edu

Q19

~Code~
340

```
40  int main() {
41      bst b;
42      add_node(&b, 4);
43      add_node(&b, 2);
44      add_node(&b, 3);
45      return 0;
46  }
```

```
typedef struct bst {
  struct node *root;
} bst;
```

Is a destructor called for bst?

```
40  int main() {
41      bst b;
42      add_node(&b, 4);
43      add_node(&b, 2);
44      add_node(&b, 3);
45      return 0;
46  }
```

clicker.cs.illinois.edu

Q20

~Code~
340

```
typedef struct bst {
  struct node *root;
} bst;
```

```
40  void bst_pop_all(            ){
41      
42
43
44
45
46
47
48  }
49
50  void bst_destruct(           ){
51      
52  }
```

```
typedef struct bst {
  struct node *root;
} bst;
```

```
typedef struct node {
  struct node *left;
  struct node *right;
  int datum;
} node;
```

C without the C++

LGs:

- Be able to read and understand C code
- Be able to write C code from scratch

1. Review
2. Dynamic memory
3. Using arrays
4. C-strings
5. Structs
6. BST example

Updates

1. MP 1.1, MP 1.2

1. Environment check-off

1. HW 2 is out

1. Collaboration time/office hours today